SWEN 563 / CMPE 663 / EEEE 663 — Real-Time & Embedded Systems

Project 3 — AL-24 "AmbuLog": Ambulatory Vitals Logger

Pocket Instrumentation Suite · Instrument #3 (STM32 HAL + CubeMX + FreeRTOS)

Assigned: Session 11 (Wed Sep 30) · **Due:** Session 14 (Wed Oct 14) — demo, memo + code due in class
Weight: 10% of the course grade · Individual project **Board:** STM32WB5MM-DK · **Framework: STM32CubeMX-generated HAL project with the FreeRTOS middleware (CMSIS-RTOS2 / CMSIS_V2)** — same CMake + Ninja + VS Code workflow as P2

> **What changes at P3:** P2 put all of the work in one cooperative super-loop. A single thread of control that you wrote by hand serviced the sensors, the display, and the console in turn, one time in each loop. That model has a ceiling, and P3 reaches it. A vitals logger must **sample on a fixed schedule that a slow flash write cannot make late**. The logger must also **let a clinician download the record while the device continues to record**. One loop cannot do these two things, because the display stalls the sampler or the download stalls the logger. A real-time operating system is the solution to this problem. You divide the device into **independent tasks**, and you connect the producer to the consumer with a **queue**. You put a **mutex** at the one position where two tasks contend for one resource. The HAL and the `.ioc` do not change, but a scheduler sits below them. S0 and P1 taught the registers, and P2 taught the HAL. P3 teaches what operates the HAL when the device must do more than one thing at the same time.

---

1. Scenario

The most important physiological events occur when no clinician is present. A resting ECG in a clinic records about forty seconds of a heart that misbehaves for four seconds a day. A single bedside vitals check gives one sample of a patient who deteriorates between rounds. Ambulatory monitoring exists to answer that problem.

The portable recorder of Norman Holter (commercialized **1962**) recorded the heart **continuously for 24–48 hours**. The record then holds the transient, intermittent event that occurs between visits (Merck Manual, *Ambulatory Electrocardiography*).

Adhesive single-lead patch recorders (the iRhythm Zio) extend continuous recording to **up to 14 days**. The longer window measurably changes what a clinician finds. In a 146-patient study the patients wore both devices at the same time. The 14-day patch detected **96 arrhythmia events against 61 for the 24-hour Holter (P < .001)**. The Holter missed 36 of those events, and the patch missed 1 (Barrett et al., *Am J Med* 2013).

P1 and P2 taught the same lesson. An average value or a spot check hides the individual event, and the continuous timestamped record shows it. In this project the record *is* the instrument.

Measurement over time is now a reimbursed standard of care, not a research luxury. CMS pays for remote physiologic monitoring under CPT **99453 / 99454 / 99457 / 99458**.

The device-supply code (99454) is billable only when the device transmits **at least 16 days of data within a 30-day period** (per CMS RPM billing guidance — *verify against MLN901705 before quoting as regulatory fact*). The payer explicitly pays for **duration and completeness of capture**, and thus for a logger that runs for days *without gaps*.

These records are clinical evidence, and regulation controls their integrity. FDA **21 CFR Part 11.10(e)** requires "secure, computer-generated, time-stamped audit trails" and states that **"record changes shall not obscure previously recorded information."** The timestamp on every sample and the append-only, tamper-evident log format in this project mirror a regulatory expectation for electronic clinical records.

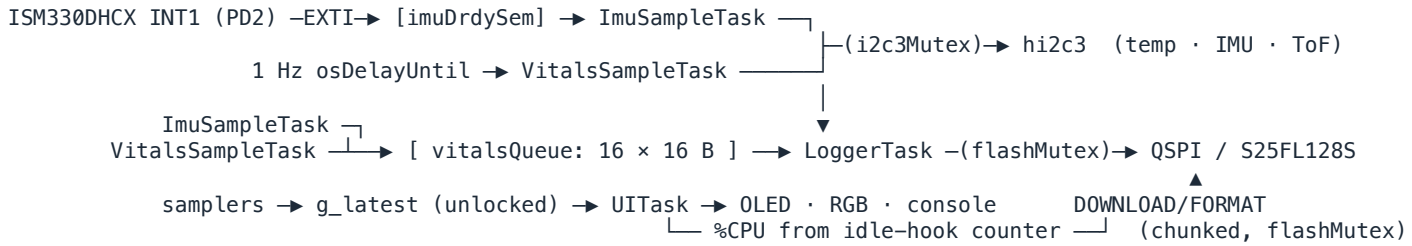
You are the firmware engineer for the **AL-24 "AmbuLog"** pilot: a pocket/bedside multi-channel vitals logger built entirely on the WB5MM-DK. It periodically samples three channels.

- **Interface temperature** — the STTS22H tracks skin/interface warming, an early marker of tissue stress (thermographic pressure-injury work uses a **>±1.1 °C** differential as an abnormal threshold — Frontiers in Public Health, 2025).
- **Activity & orientation** — the ISM330DHCX accelerometer logs movement and body posture. Wearable actigraphy is a validated mobility/immobility and fall-risk channel in acute care (JMIR Aging, 2025).
- **Presence / proximity** — the VL53L0X time-of-flight sensor reports whether the patient (or a limb) is present in front of the pod. This channel is the "is anyone there" gate of the logger.

The device timestamps each reading, appends it to the **16 MB QSPI NOR flash** on the board as a tamper-evident log, and shows a live readout on the OLED. It lets a clinician **download the whole record over the USB-VCP link while the device keeps recording**. The commercial analogs are the **iRhythm Zio** patch, the **VitalConnect VitalPatch** biosensor, and the research-grade **Philips Actiwatch** actigraph.

What the AL-24 actually does — the core function

> **The AL-24 samples the vitals channels, each channel on its own periodic schedule. It timestamps each sample and appends it to the tamper-evident onboard flash-log. At the same time it serves a live display and an on-demand clinician download over UART. It drops no sample, and it never stalls the recorder. It reports its own CPU utilization, so that an operator can see that the device is not silently overloaded.**



Two arrows make this an RTOS project and not a super-loop. The first arrow is **the queue**, because a slow page-program does not stall a sampler. The second arrow is `flashMutex`, because a clinician download and the recorder touch the same flash and must take turns. The other parts are the P2 parts that you know, divided across tasks.

---

2. Learning objectives

By completing this project you will demonstrate that you can:

1. Enable **FreeRTOS in STM32CubeMX** (CMSIS-RTOS2 / CMSIS_V2, `heap_4`), with the mandatory move of the **HAL timebase off SysTick onto a spare timer**. Then explain *why* the RTOS must own SysTick.
1. Decompose a device into **cooperating tasks** with defensible **priorities** and **periods**. Use `osDelayUntil` for jitter-free periodic sampling, not `osDelay`. Use `uxTaskGetStackHighWaterMark` to set the stack dimensions from measurement and not from guesswork.
1. Use a **message queue** to decouple a fixed-rate **producer** (the samplers) from a **consumer** (the logger). The time that the logger needs for one record varies, but no sampler blocks on flash.

1. Give a genuinely **shared resource the protection of a mutex**, and name accurately what breaks without the mutex. Connect the mutex to **priority inversion and inheritance** (L13). Also justify in writing the shared data that you deliberately **left without a lock**, because not all data needs a lock.
1. Hand work from an **ISR to a task** with the RTOS-correct procedure. A data-ready interrupt gives a **binary semaphore** and yields. The task does the I²C read. Interrupt context has no bus transactions and no non-`FromISR` API (L15).
1. Design an **append-only, timestamped flash-log** on a QSPI NOR part. The flash-log has a fixed record format with an integrity check and page-batched writes. The erase strategy keeps a **650 ms sector erase off the hot path**, and the **overflow** behavior is specified. The result is an audit record fit for a compliance review.
1. Measure the **CPU utilization of the device with the FreeRTOS idle hook** (`vApplicationIdleHook`). Report it live. Know what that aggregate number tells you and what it does not tell you.
1. Use the component drivers and course modules from P2 again, **unchanged in silicon but re-disciplined for concurrency**. Keep the full project able to **survive CubeMX regeneration** (R32).

---

3. The instrument — modes of operation

Mode	Purpose
POST	Power-on self-test, run **before the scheduler starts**. It brings up I²C3 / SPI1 / USART1 / QUADSPI. It identifies each sensor by ID (STTS22H `0xA0`, ISM330DHCX `0x6B`, VL53L0X model ID) and verifies the flash by its JEDEC ID. It reports each device found or missing **by name** to console and OLED. It prints the firmware version, the thresholds, and the **idle-count calibration baseline**. POST names a missing device and fails safe (no logging claims).
RUN (default)	The scheduler runs. All sampler tasks are active. `LoggerTask` drains the queue to flash. `UITask` paints the live readout, drives the RGB alert LED, and refreshes the **%CPU** figure. The device timestamps and logs all samples.
CONFIG	Console-gated parameters, non-blocking: view and set the sample period and the alert thresholds, `FORMAT` the log (the one position where a sector erase runs), `STATUS`, start and stop a logging session without reset. RUN sampling can continue below, and this depends on the command.
DOWNLOAD	Streams the committed log over the VCP to the terminal of a clinician. **RUN does not stop.** The recorder continues to sample and to log while the reader drains the flash in bounded chunks. This concurrency is the main synchronization lesson of the project.

Buttons. B1 cycles the OLED pages. The sequence is `LIVE` (temp / posture / proximity + %CPU) → `LOG` (record count, coverage %, LOG-FULL/OVERFLOW condition) → `TASKS` (per-task stack high-water marks, queue depth). The **LIVE and LOG renderers ship in the starter** (`ui_pages.c`, §6), because the formatting is plumbing and not the lesson. The **TASKS page is yours to write**, and it is the R5 evidence surface.

B2 is a session marker. It writes a timestamped `EVENT` record into the flash-log, which records that the clinician pressed the button at that moment. The step is deliberate, and the device logs it.

---

4. Functional requirements

Requirement language follows RFC 2119 (**SHALL** = mandatory, demonstrated at the bench). Demo-scaled defaults are in §7.3. All thresholds below are CONFIG parameters, not constants. "After the scheduler starts" means all code that the device reaches after `osKernelStart()` starts the scheduler.

Tasks & scheduling

Id	Requirement
R1	The application SHALL run on **FreeRTOS through CubeMX (CMSIS-RTOS2 / CMSIS_V2,** `heap_4`**)**, with the **HAL timebase moved off SysTick onto a basic timer** (TIM16 or TIM17 — record the timer that CubeMX assigns, because the graders ask you for it at the bench). SysTick belongs to the RTOS tick. `HAL_GetTick()` runs on the dedicated timer.
R2	The device SHALL be built from **at least four application tasks** with distinct, documented responsibilities and priorities: a data-ready-driven **IMU sampler**, a periodic **vitals sampler** (temp + ToF), a **logger** (queue consumer / flash owner), and a **UI/console** task. You justify the existence and the priority of each task, relative to the others, in one line for each task on the task-table appendix of the memo (§10), and you defend them at the bench Q&A.
R3	The periodic vitals sampler SHALL use `osDelayUntil` (absolute wake time) for its pace, not `osDelay`, so its period does not drift with its own execution time.
R4	The **ISM330DHCX data-ready interrupt (INT1 → PD2 / EXTI2)** SHALL wake the IMU sampler through a binary semaphore. The IMU sampler SHALL NOT poll the IMU on a timer.
R5	Task stacks SHALL be **right-sized with** `uxTaskGetStackHighWaterMark` during bring-up (reported on the `TASKS` OLED page), not left at guessed values. `configCHECK_FOR_STACK_OVERFLOW` SHALL be enabled (method 2) with a logging `vApplicationStackOverflowHook`.

Synchronization & shared resources

Id	Requirement
R6	**PRIMARY shared resource — the flash.** All QSPI / S25FL128S transactions (page-program, header write, log read, erase) SHALL be serialized by a single `flashMutex` created with priority inheritance. Be ready to name the two failures that this prevents at the bench Q&A (it is the scripted question): a **torn read** (the download streams a region mid-program), and a **peripheral wedge** (two tasks race for the one `hqspi` command/status engine).
R7	**SECONDARY shared resource — the I²C3 bus.** All `HAL_I2C_Mem_Read/Write` calls on `hi2c3` (temp, IMU, ToF all share SCL=PB13 / SDA=PB11) SHALL be serialized by an `i2c3Mutex` created with priority inheritance. The HAL I²C driver is not reentrant. Two tasks that issue transactions on one handle corrupt the transfers of each other.
R8	A **producer→consumer queue** SHALL carry timestamped sample records from the samplers to the logger. A sampler SHALL NOT call the flash driver directly, and a queue-put SHALL NOT block a sampler beyond one sample period. Queue-full events SHALL be counted and reported.

Id	Requirement
R9	The clinician `DOWNLOAD` SHALL acquire and release `flashMutex` **for each bounded chunk** (≤ 16 pages / 4 KB), and never hold it for the whole command. `LoggerTask` thus gets fair, bounded windows to append between chunks, and RUN never stalls.
R10	No ISR SHALL call a non-`FromISR` RTOS API. ISRs SHALL do only a semaphore-give (or ISR-safe queue-put with timeout 0) and `portYIELD_FROM_ISR`. All bus/flash/display/log work runs in task context.
R11	The device SHALL keep at least two pieces of shared data **deliberately without a lock** — the "latest sample" snapshot that the UI reads, and the idle-hook counter. The memo SHALL **justify why each needs no lock** (single-writer/single-reader with harmless staleness, a naturally atomic 32-bit access) and contrast them with the flash, where concurrent access means corruption.

Sampling & sensors

Id	Requirement
R12	Interface **temperature** (STTS22H, °C → tenths) and **proximity** (VL53L0X, range in mm) SHALL be sampled at a configurable period (default **1 Hz**). **Activity/orientation** (ISM330DHCX accel, mg) SHALL be sampled on its data-ready interrupt at its configured ODR (default ~12.5 Hz).
R13	Each sample SHALL be **timestamped with** `HAL_GetTick()` **at the moment of acquisition** (not at enqueue and not at write), so the log shows when the device took the reading.
R14	The IMU sampler SHALL derive a coarse **posture class** (for example UPRIGHT / SUPINE / LEFT / RIGHT / PRONE from the gravity vector) and log it alongside the raw axes. Floating-point math is permitted internally, but logged and printed values stay **integer** (newlib-nano, as in P1/P2).
R15	An out-of-range or failed sensor read SHALL be **flagged in the record** (a fault bit), never fabricated or silently substituted. A persistent sensor fault SHALL be surfaced on the OLED and console.
R16	The ToF and temperature reads and the IMU read all share I²C3 and SHALL each take `i2c3Mutex` for the duration of their transaction only (R7) — held for the transaction, not for surrounding math.

Storage & the audit log

Id	Requirement
R17	Log records SHALL be a **fixed 16 bytes** — `timestamp_ms (u32)`, `channel (u8)`, `flags (u8)`, `payload[8]`, `crc16 (u16)` — so exactly **16 records fill one 256-byte flash page** and the record size divides both flash sector sizes. The CRC-16 SHALL cover the bytes of the record itself. A record that fails CRC on download SHALL be **flagged, never silently repaired or dropped** (21 CFR Part 11 integrity).

Id	Requirement
R18	`LoggerTask` SHALL **batch 16 records into a 256-byte RAM page and issue exactly one QSPI page-program for each full page** (with a partial-flush after a bounded idle interval so the last few samples are not stranded). Page-program time is ≤ 750 µs worst case. The mutex is held for the program, not longer.
R19	A **sector erase (≤ 650 ms) SHALL occur only inside the operator-initiated `FORMAT` command** (CONFIG mode), which pre-erases the data region before a session. The steady-state append path SHALL NOT erase — it only page-programs into flash that is already erased. A bench Q&A item (§9) asks why an erase on the hot path, inside `flashMutex`, is a real-time defect.
R20	A **header sector** (a 4 KB parameter sector) SHALL persist the `write_pointer`, `record_count`, `session_id`, and `log_full` flag of the log across reset, updated by page-program at a rotating slot in the sector (to spread wear before an erase). On boot the device SHALL recover its position from the header.
R21	On data-region exhaustion the device SHALL set **LOG FULL**, stop appending, and report an **OVERFLOW** count — it SHALL NOT silently overwrite older records (this mirrors "record changes shall not obscure previously recorded information"). True circular overwrite is a graduate alternative (§8), not base behavior.
R22	The log SHALL live on the **16 MB external QSPI flash** — be ready to justify at Q&A why the log lives on external flash (capacity: 16× the **1 MB internal flash** of the module, which must hold the application + BLE stack image).

Download & console (the clinician interface)

Id	Requirement		
R23	`DOWNLOAD` SHALL stream the committed records (record 0 → write-pointer) over the VCP as a **documented, parseable text/CSV format** (one line for each record: timestamp, channel, decoded value, flags, CRC-OK). The PuTTY session log of a download **is** the audit record, and you submit it with the memo.		
R24	`DOWNLOAD` SHALL run **concurrently with RUN sampling and logging** and SHALL NOT cause a dropped sample (bounded by queue depth in R8/R9). You demonstrate this live (D5).		

Id	Requirement		
R25	The console SHALL be **non-blocking** and operable while RUN continues. Command dispatch uses the **provided command-table parser** (§6). You write the **handlers** (HELP, STATUS, `MODE RUN	CONFIG, SET PERIOD <ms>, SET THRESH TEMP <cC>	DIST <mm>, FORMAT — which SHALL still require its confirmation token — DOWNLOAD [since <ts>], CPU) and the **non-blocking 50 ms console tick integration** into your task set — the one new skill here. The device SHALL echo all accepted parameters as a logged EVENT`.
R26	STATUS SHALL print a one-screen summary: mode, uptime, per-channel last value, record count, coverage %, LOG-FULL/OVERFLOW condition, queue depth / high-water, per-task stack high-water, and current **%CPU**.		

Idle-hook performance monitor

Id	Requirement
R27	`configUSE_IDLE_HOOK` SHALL be enabled and `vApplicationIdleHook()` SHALL do nothing but **increment a counter** — no blocking, no OS calls, no `printf`.
R28	The device SHALL calculate the **%CPU load one time each second** from the idle count against a **boot-measured calibration baseline** (`ulIdleCountMax`, printed at POST), and display it on the OLED `LIVE` page and in `STATUS/CPU`. State the limitations of the method at Q&A (aggregate only, and recalibrate on a clock change or an optimizer change). The %CPU **rises visibly during a `DOWNLOAD` or `FORMAT`**, and you demonstrate this live (D5/D7).

Framework discipline (this is a FreeRTOS + CubeMX project)

Id	Requirement
R29	After `osKernelStart()`, there SHALL be no `HAL_Delay()` and no busy-wait — all waiting is `osDelay` / `osDelayUntil` / blocking on a queue or semaphore. The graders will read your tasks. (Pre-scheduler init code, for example `oled_init`, can use `HAL_Delay`.)
R30	ISRs/callbacks SHALL be minimal (R10). An `HAL_I2C_*` or `HAL_QSPI_*` call in an ISR/callback, or a mutex take from ISR context, is an automatic finding.
R31	The idle hook, stack-overflow hook, and the timebase-timer ISR SHALL be left as CubeMX generates them (priority and wiring). Do not hand-lower the NVIC priority of the timebase timer, and do not touch `xPortSysTickHandler`. These changes break `HAL_GetTick()` under load.

Id	Requirement
R32	The project SHALL be **buildable after a clean CubeMX regeneration**: all application code lives in `USER CODE` sections or your own `.c/.h` files listed in `CMakeLists.txt`. Code review and the §11 forfeit enforce regeneration survival. The graders grade the live regen ritual one time, at P2 D1.

---

5. Design constraints

1. **Two ticks, one clock.** The **RTOS tick** (SysTick) schedules the tasks. The **HAL tick** (`HAL_GetTick()`, on TIM16/17) is your **wall clock and the source of the timestamp in each record**. Do not invent a second timing authority. Know why there are two ticks and what each tick is for.

1. **Reporting stays integer.** You can use float math in the posture and threshold logic. The `printf` and OLED formatting use integer tenths (`%d.%d`), because newlib-nano has no float `printf`, the same as P1/P2.

1. **Provided drivers, re-disciplined for concurrency.** The starter gives you the course OLED, console, and RGB modules and the STTS22H / ISM330DHCX component drivers (HAL, from P2). **The silicon logic stays unchanged, but the code that calls it does not.**

Each shared-peripheral access (SPI1/OLED, USART1/console, I²C3/sensors) runs under a task, and under a mutex where two tasks can get to it. Read the modules. Find where they block (OLED SPI transmit ~ms, RGB ~11 ms bit-bang), and select the task that owns each one.

1. **Do not use the full board BSP from ST** (`Drivers/BSP/STM32WB5MM-DK`). As in P2, its bus layer re-inits peripherals on private handles and collides with the CubeMX-generated code that you must keep (R32). The QSPI flash and ToF drivers ship as thin **components** in the starter (§6), the same model as the P2 sensors.

1. **The IR LED shares QSPI IO0 (PB9).** The infrared LED on the board uses the same pin as flash data line IO0. This project does not use the IR LED, but the constraint exists (UM2825 §7.8.5).

1. **Do-not-touch `.ioc` assignments** carried from P2 (OLED SPI1, VCP USART1, RGB PA7/PH1, I²C3, buttons, ST-LINK) stay as generated. P3 **adds** QUADSPI (PA3/PD3/PB9/PD5/PD6/PD7), the ToF wiring on I²C3, the FreeRTOS middleware, and the TIM16/17 timebase.

---

6. Starter package & the CubeMX template

The starter is `P3_Logger_Starter/` — a working FreeRTOS project that you flash on day one. The smoke test has two blinking tasks, an OLED banner from a UI task, console echo, a flash JEDEC-ID read, and a live %CPU number from the idle hook.

The course makes the starter with the procedure that it teaches: the P2 `.ioc` opened in CubeMX → **FreeRTOS middleware enabled (CMSIS_V2) → timebase moved to TIM16/17 → QUADSPI added** → ToF bound on I²C3 → code generated (CubeMX 6.17.0, FW_WB V1.24.0, CMake toolchain). Code review checks that the project survives a regeneration from the `.ioc` (R32). Demo row D0 opens with POST.

What the `.ioc` adds over P2 (open it in CubeMX and check it — this is your map):

Peripheral / setting	Purpose	Configuration
FREERTOS	The scheduler	Interface **CMSIS_V2**, `heap_4`. `USE_IDLE_HOOK=1`, `CHECK_FOR_STACK_OVERFLOW=2`. You can create the tasks, the queue, and the mutexes in code (recommended) or in the `.ioc`
SYS → Timebase Source	HAL tick off SysTick	**TIM16** (or the timer that CubeMX assigns — record it). SysTick drives the RTOS tick only
QUADSPI	S25FL128S NOR flash log	CLK=PA3, NCS=PD3, IO0=PB9, IO1=PD5, IO2=PD6, IO3=PD7. Indirect mode for page-program and erase
I²C3 (unchanged)	temp + IMU + **ToF**	PB13=SCL / PB11=SDA · 100 kHz. The VL53L0X (0x29) joins the two P2 sensors on this bus
GPIO EXTI (from P2)	IMU data-ready	PD2 = ISM330DHCX INT1 (EXTI2) — the wake source of the sampler

These parts are deliberately missing: radio, USB-device, ADC, DSP. They come in P4/P5.

Driver architecture (the README gives the full description):

- `Drivers/Components/stts22h`, `Drivers/Components/ism330dhcx` — **carried from P2 unchanged.**
- `Drivers/Components/vl53l0x` — **provided ToF driver**, vendored and black-box. You use `RegisterBusIO` / `Init` / `ReadID` / `GetRange`. The VL53L0X has no public register map in its datasheet, and the full API from ST is a large middleware layer. You integrate it, and you do not reverse-engineer it.

(See the open decision in §6.1 below — this driver is the one starter piece that the instructor must finalize before release.)

- `Drivers/Components/s25fl128s` — **provided QSPI flash driver**: JEDEC-ID read, sector erase, page-program, read, and status-poll against the pin map above. Thin HAL-level code in the course style, not the ST BSP.
- `App/vitals_bus.c` — the P2 I²C3 IO layer, **extended with a third binding for the ToF** on the same five callbacks (no new IO-layer functions — one more `RegisterBusIO`).
- `App/oled.c`, `App/console.c`, `App/rgb_led.c` — provided course modules. Use the silicon logic again, and put the task-and-mutex discipline of §5.3 around the call sites.
- `App/cmd_table.c` — **provided command-table parser** (name → handler → help text over the console line-reader): non-blocking line assembly, tokenizing, and dispatch. **You write the R25 handlers, and you select where the ~50 ms console tick lives in your task set** — the parser gives the plumbing, and the integration is the skill.
- `App/ui_pages.c` — **provided LIVE and LOG page renderers** (formatting over `oled.c`, with B1 page cycling). The **TASKS page is a stub — you write it** (per-task stack high-waters with `uxTaskGetStackHighWaterMark`, queue depth / high-water — the R5 evidence path).
- `App/log_store.c` (skeleton) — the append-only abstraction for the records and the header over the flash driver: `log_format`, `log_append`, `log_read_record`, `log_status`. This module keeps the flash-mutex discipline in one position, and you complete it. The **header-sector struct and rotating-slot layout ship as provided definitions** in the skeleton. **Recovery-on-boot and the slot-rotation update are yours** (R20).

- `App/tasks.c` (skeleton) — task entry points, the queue, the two mutexes, the semaphore, and the idle hook, with the wiring stubbed and commented. This is the module that you make into working code.
- `Core/`, `cmake/stm32cubemx/`, `Middlewares/Third_Party/FreeRTOS` — CubeMX/kernel territory: user code only in `USER CODE` fences (R32).

6.1 One open decision for the instructor (flagged, does not block the spec)

The **VL53L0X integration path** is the one item to finalize before the starter ships. Alternative (a) is to **vendor the VL53L0X component/BSP driver from ST** in STM32CubeWB as a black box. This alternative is the recommended one, because students use four functions, the same as the P2 sensor model. Alternative (b) is to put **UM2039** and the API source from ST into the collateral, so that a course driver can be thinner.

The datasheet in `Collateral/05` has **no register map**. A "poll a range register" method is thus *not* vendor-supported, and the course does not give it to students as fact. The requirements of this specification (R12/R16) apply to the *interface* of the driver, so the two alternatives satisfy them. This is a packaging selection for the starter build, not a change to the specification.

---

7. Hardware & bench setup

7.1 No external stimulus

Everything that the AL-24 needs is on the board.

Temperature: cup a hand over the STTS22H (U4) to cause a trend.

Activity/posture: lift the board. Tilt it. Set it on its edge. The IMU logs the movement and the posture class.

Proximity: wave your hand in front of the ToF window (U6). As an alternative, point the board at a wall at a fixed distance.

No signal generator and no scope are necessary. The physical world is the stimulus, as in P2.

7.2 The bench demo without patience

Clinical timescales are too long for a demonstration, and a multi-hour flash fill is also too long. `FORMAT` **then** `SET PERIOD 50` fills the demo-scaled data region in seconds. You can thus show **LOG FULL**, **DOWNLOAD-during-RUN**, and the **%CPU spike** at your desk before demo day, with no external equipment and with no wait.

7.3 Demo scaling (clinical ↔ bench)

Each parameter below is a CONFIG parameter, and these are the demo defaults:

Parameter	Clinical reality	Demo default
Continuous recording window	24 h – 14 days (Holter → patch)	a studio session (minutes)
Vitals sample period (temp, ToF)	seconds–minutes	**1 s**
IMU activity rate	event/continuous	**~12.5 Hz on data-ready**
Log data region	days of capacity (16 MB)	**2 × 64 KB sectors (~8k records)** so LOG FULL is reachable in a demo
Alert: temperature rise	>±1.1 °C interface differential	**+2.0 °C over a running baseline**
Alert: proximity	presence/bed-exit	**object < 100 mm**
Idle-count window	—	**1 s**

Your decision memo (§10) SHALL include a paragraph. The paragraph maps each demo value to its clinical counterpart, and cites the basis for the recording-duration and temperature-differential figures.

---

8. Graduate-student requirement (663 sections — choose one)

Students in a **graduate section (CMPE 663 / EEEE 663)** SHALL implement **one** alternative, demonstrate it live (row **DG**), and document the design and the results in the GR section of the memo. Undergraduate (563) students are not graded on this section. A team with students from the two sections has the requirement.

GR-A — Priority inversion, measured with inheritance and without it. Instrument the wait of `LoggerTask` on `flashMutex`. Timestamp (with `HAL_GetTick()` or the DWT cycle counter) the instant when the task *requests* the mutex and the instant when the task *gets* the mutex. Do this while a `DOWNLOAD` runs in the low-priority `UITask`, and while the two samplers supply their usual medium-priority work.

Give a console toggle `INHERIT ON|OFF`. `ON` puts a **priority-inheriting mutex** on the flash. `OFF` puts a **plain binary semaphore** on the flash (guaranteed non-inheriting — this avoids the fact that CMSIS-RTOS2 `osMutexNew` can always inherit and can ignore the attribute bit).

Demonstrate this live. With inheritance **OFF**, the samplers starve the holder of the download, and the grant latency of `LoggerTask` grows without bound. You can see the result as a gap in the downloaded record sequence, or as a queue-overflow count. With inheritance **ON**, the maximum stall is one download chunk, and the gap disappears. Report a **latency histogram** for the two conditions.

(This ties directly to L13, R6, and demo D5.)

GR-B — Ring overwrite with download-safe wraparound. Replace the "stop at LOG FULL" behavior of R21 with true circular overwrite. At region-full, `LoggerTask` uses the oldest block again, but it SHALL NOT overwrite a block that a `DOWNLOAD` holds at that moment.

Monitor the block of the reader under `flashMutex`. Defer the overwrite, and count it in its own `OVERWRITTEN` counter. Do not corrupt a download that is in operation.

The GR section of the memo SHALL discuss the explicit tension between circular overwrite and 21 CFR Part 11, "shall not obscure previously recorded information". It SHALL also justify overwrite as a documented, bounded tradeoff. *(As the second alternative, the instructor can offer* `configGENERATE_RUN_TIME_STATS` *for a per-task CPU report next to the idle-hook aggregate.)*

---

9. Demonstration requirements (the demo sheet)

Row	You demonstrate	Pass looks like
D0	**POST**	Per-device ID checks (STTS22H / ISM330DHCX / VL53L0X / S25FL128S) PASS on console + OLED. Idle baseline printed
D1	RUN live sampling	OLED `LIVE` shows temp / posture / proximity that update at their rates. `STATUS` agrees. Records accumulate (R2, R12, R13). Pass reads the **student-written record path**: values that update at their configured rates AND a `STATUS` record count that increments (the provided renderers display the values, and your data flow feeds them)

Row	You demonstrate	Pass looks like
D2	Periodic integrity	`TASKS` page shows the vitals sampler that holds its period under load (osDelayUntil), and stack high-waters that stay stable (R3, R5) (the TASKS page itself is student-written)
D3	ISR→task handoff	Movement of the board produces IMU records. The EXTI handler only gives a semaphore (shown in code review) (R4, R10)
D4	Idle-hook %CPU baseline	Quiet RUN sits at a low, stable %CPU on OLED + console (R27, R28)
D5	`DOWNLOAD` **while RUN keeps logging** — the central row	The log streams over UART. %CPU visibly rises. **Sampling never stops and no record is dropped.** Downloaded sequence + CRC intact (R6, R9, R24)
D6	Force **LOG FULL** (small region / fast period)	Device reports LOG FULL / OVERFLOW, stops appending cleanly, no crash or corruption (R21)
D7	`FORMAT` in CONFIG	Multi-second bulk pre-erase completes with a visible %CPU spike. Fresh log at record 0. RUN resumes (R19, R25)
D8	Alert + session marker	Warm the temp sensor / bring an object close → RGB shifts to alert, and the device logs it. **B2** writes a timestamped `EVENT` marker that you can see in the download (R15, R25)
DG	*663 only:* GR-A inversion histograms or GR-B ring/stats	As given in §8, graded like any other row
Q&A	"What breaks if you delete `flashMutex`?" · "Why does the latest-sample snapshot need no lock but the flash does?" · "Why is the UI task the lowest priority?" · "Why would an erase on the hot path, inside `flashMutex`, be a real-time defect?" (R19)	Precise answers: torn-read + peripheral-wedge. Single-writer with harmless staleness, against read+write corruption. UI is the deferrable work. A ≤650 ms erase held under `flashMutex` stalls the logger for hundreds of sample periods, and thus the erase belongs in FORMAT

During all rows the console stays responsive, and sampling never stalls the UI. The graders will `Ctrl-F` your submission for `HAL_Delay` after scheduler start (R29) and for bus or flash operations in ISRs (R30).

---

10. Deliverables

1. **Code** — the project as built (buildable through `cmake --preset Debug` and after CubeMX regeneration), your application isolated in user sections or your own files, FreeRTOS enabled through the `.ioc`.
1. **Decision memo** (PDF, 2–3 pages) with ONLY these parts:

- (a) the **two deliberately unlocked variables and why each needs no lock** (R11) — single-writer / single-reader with harmless staleness, and a naturally atomic 32-bit access — contrasted with the flash, where concurrent access means corruption. This is the one **when-NOT-to-lock** written argument in the course, and the graders grade it as such.

- (b) the **§7.3 demo-scaling table with clinical citations** — the paragraph that maps each demo value to its clinical counterpart.
- (c) 663: the GR section from §8 (the design and the results).

Auto-generated appendix (attached, not prose): the task table — name / priority / period / stack high-water — with **one line of priority rationale for each row** (R2). Print it from code, or transcribe the `STATUS` screen. One line for each row is sufficient.

The course removed the old long-form report from the deliverables. That report gave the task decomposition, the synchronization design, the flash-log design, and the %CPU method. The graders now grade these items where they live: at the bench (D1–D8 and the scripted Q&A) and in code review.

1. **The demo-day PuTTY logs** — the `DOWNLOAD` transcript and the session log, unedited, as the audit record (R23).

---

11. Grading

Component	Weight
Bench demo D0–D8 (each row pass/partial/fail)	**60%**
Code review: RTOS discipline (tasks, mutex/queue correctness, ISR hygiene, R29–R32), structure, readability	**25%**
Decision memo (incl. unlocked-variable justification + clinical mapping)	**15%**

663 sections: the demo weight includes **DG** (D-rows + DG share the 60%), as in P1/P2.

Hard findings: `HAL_Delay` or busy-wait after scheduler start, a bus or flash operation in an ISR, or a mutex taken from ISR context caps the code-review component at half. A download that stalls the recorder (fails D5) forfeits that row and the corresponding code-review line. A project that does not survive a clean regeneration (R32 — checked at code review on suspicion) forfeits the code-review framework-discipline line.

---

12. References

Clinical / regulatory

- Barrett PM, Komatireddy R, Haaser S, et al., "Comparison of 24-hour Holter Monitoring with 14-day Novel Adhesive Patch Electrocardiographic Monitoring," *Am J Med* 2013;126(2):95-97 (96 vs 61 events, P<.001).
- Merck Manual, *Ambulatory Electrocardiography (Holter and Event Monitors)* — 24–48 h continuous recording. Holter monitor history (commercialized 1962).
- iRhythm Technologies — Zio extended-wear (up to 14-day) single-lead patch ECG monitor (device analog).
- CMS Remote Physiologic Monitoring — CPT **99453 / 99454 / 99457 / 99458**. The 99454 "≥16 days in 30" device-supply threshold (MLN901705 RPM billing guidance — *confirm at cms.gov before quoting as regulatory fact*).
- FDA **21 CFR Part 11.10(e)** — secure, computer-generated, time-stamped audit trails; "record changes shall not obscure previously recorded information" (eCFR, Title 21).
- Frontiers in Public Health (2025), skin interface pressure/temperature monitoring for pressure-injury prevention — abnormal interface temperature differential >±1.1 °C.

- JMIR Aging (2025), wearable devices for monitoring mobility/posture in hospitalized older adults (actigraphy as a validated mobility/immobility channel).
- Commercial analogs: iRhythm **Zio**, VitalConnect **VitalPatch**, and Philips **Actiwatch** (actigraphy).

Device / engineering

- UM2825 Rev 5 — STM32WB5MM-DK user manual (QSPI flash U8 map §7.8.3, ToF U6 & I²C3 §7.9, pin table Table 20).
- S25FL128S datasheet (Cypress/Infineon) — 128 Mbit (16 MB), 256-byte page, 4 KB/64 KB hybrid sectors. Page-program is typ 250 µs / max 750 µs. Sector erase is typ 130 ms / max 650 ms (`Collateral/05`).
- VL53L0X datasheet — I²C 0x29, 33 ms default timing budget. The ST API is a turnkey C layer with no public register map (`Collateral/05`). See §6.1 for the driver decision. AN4760 — QUADSPI interface (`Collateral/02`).
- DS11929 — STM32WB55xx datasheet (QUADSPI §3.27, up to 1 MB internal flash) · DS13252 — STM32WB5MMG module (1 MB flash / 256 KB SRAM).
- UM2442 — STM32WBxx HAL/LL manual · UM1718 — STM32CubeMX · UM1722 — STM32Cube FreeRTOS/CMSIS-RTOS development (SysTick-handler collision, timebase information) (`Collateral/03`, `Collateral/04`).
- *Mastering the FreeRTOS Real-Time Kernel* (idle hook, stack-overflow hook, run-time stats) · FreeRTOS Reference Manual v10 (`Collateral/04`).
- TLC59731 — RGB LED single-wire driver (reused from P1/P2).